Stefan Henkler E-Mail: stefan.henkler@hshl.de

Faezeh Pasandideh E-Mail: faezeh.pasandideh@hshl.de

HOCHSCHULE HAMM-LIPPSTADT

1. Software Engineering Life Cycle Models
2. Embedded System Life Cycle Models
3. Advanced Life Cycle Models & MDD
4. Process Improvement
5. Discussion & Summary
6. Bibliography

Life cycle

- A series of stages/phases… through which something (in our case a product, prototyp, …) passes during its lifetime

- Or in short: a sequence of stages/phases in the life of a system
  - It start from the need to create the system
  - It ends when the is retired and removed from service

- Different stages/phases are defined
  - E.g. Clark et al 86
    - Planing, definition, design, integration, acceptance, delivery product

First phase: Operational Need

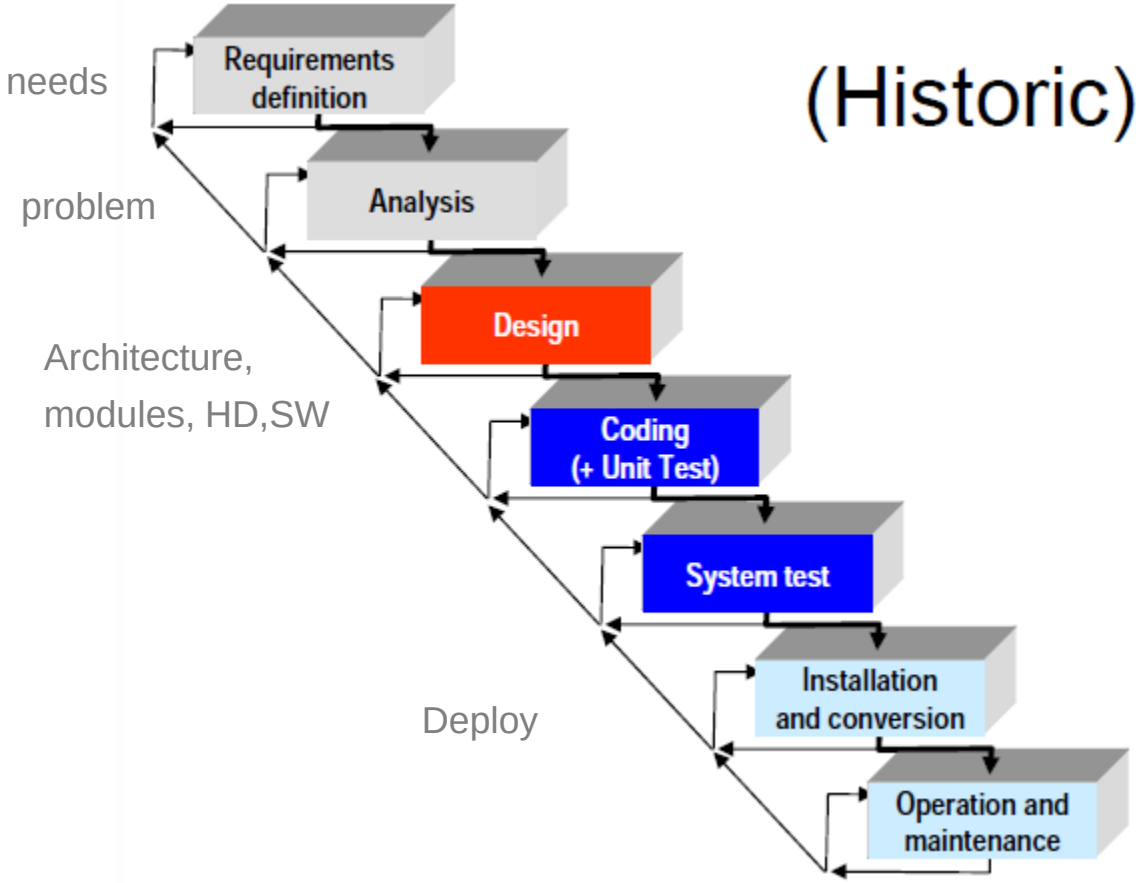
- The reason for a system to be developed
  - Why does one need this system
  - And what is needed (high level)
- Needs come from stakeholders
  - Customer,
  - Government Agency, …
- Needs origination
  - National strategic decision making
    - E.g. in form of a request of proposal (RFP)
  - Organizational strategy
    - Marketing surveys
    - Customer feedback
    - Changing environment
    - New technolgy
    - New ressources, …

Example: design needs small aircraft

Parameter	Description
Range (sufficient)	At least (≥) 200 miles, 30 min reserve fuel, to cover long distances with a safety reserve.
Efficiency (high)	≥200 passenger miles per gallon equivalent, to min consumption
Speed (qdequate)	≥ 100 mph average on each of two 200 mile flights, for practical transportation
Minimum speed (safe)	Must fly level at 55 mph in level flight without stall, power and flaps allowed, to avoid aerodynamic stall
Takeoff distance (limited)	Clear a 50-foot obstacle within 2,000 feet of runway, to enable opt from relatevilty short runways
Community noise(low)	No more than 80 dBA measured 240 feet from takeoff point, to reduce environmental impact, disturbance to peope.

6 Software Engineering Life Cycle Models

- Waterfall Model
- Prototyping
- V Model
- Spiral Model
- RUP



(Historic) Waterfall Model

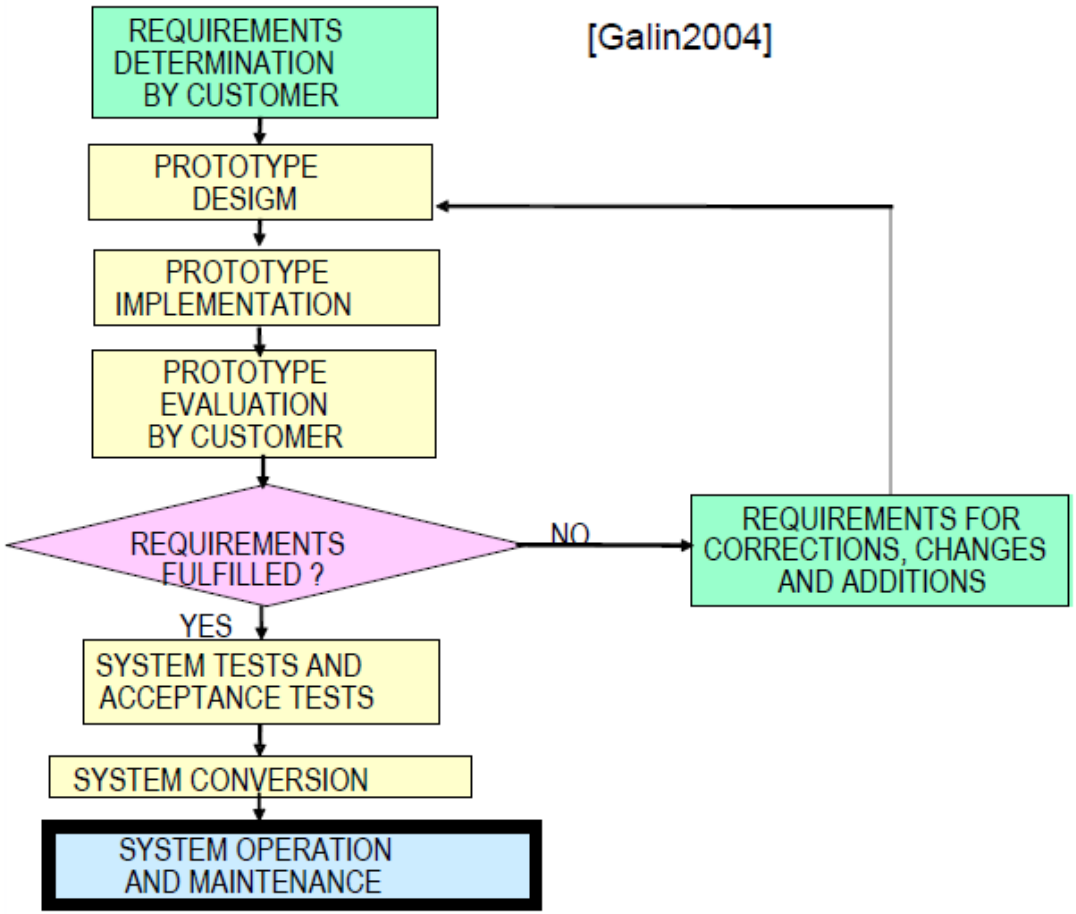
What is the big problem? Historic,1970s

Using Waterfall for HSUV:

- Week 1 define all requirements,
- Week 2 design circuits,
- Week 3 code,
- Week 4 test.

Problem: if in Week 4 you discover IR sensors cannot read gray concrete, going back costs enormous time.

9 The Prototyping Process

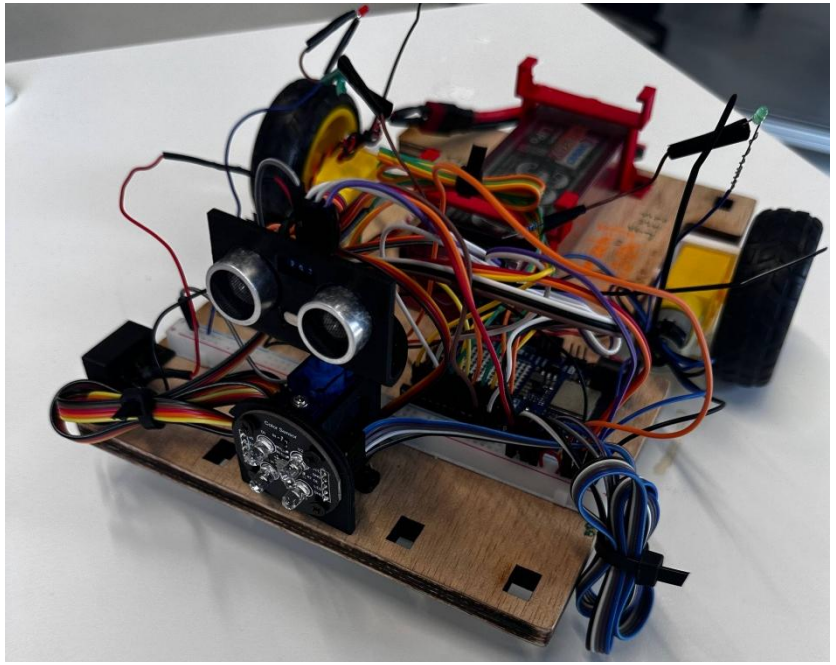


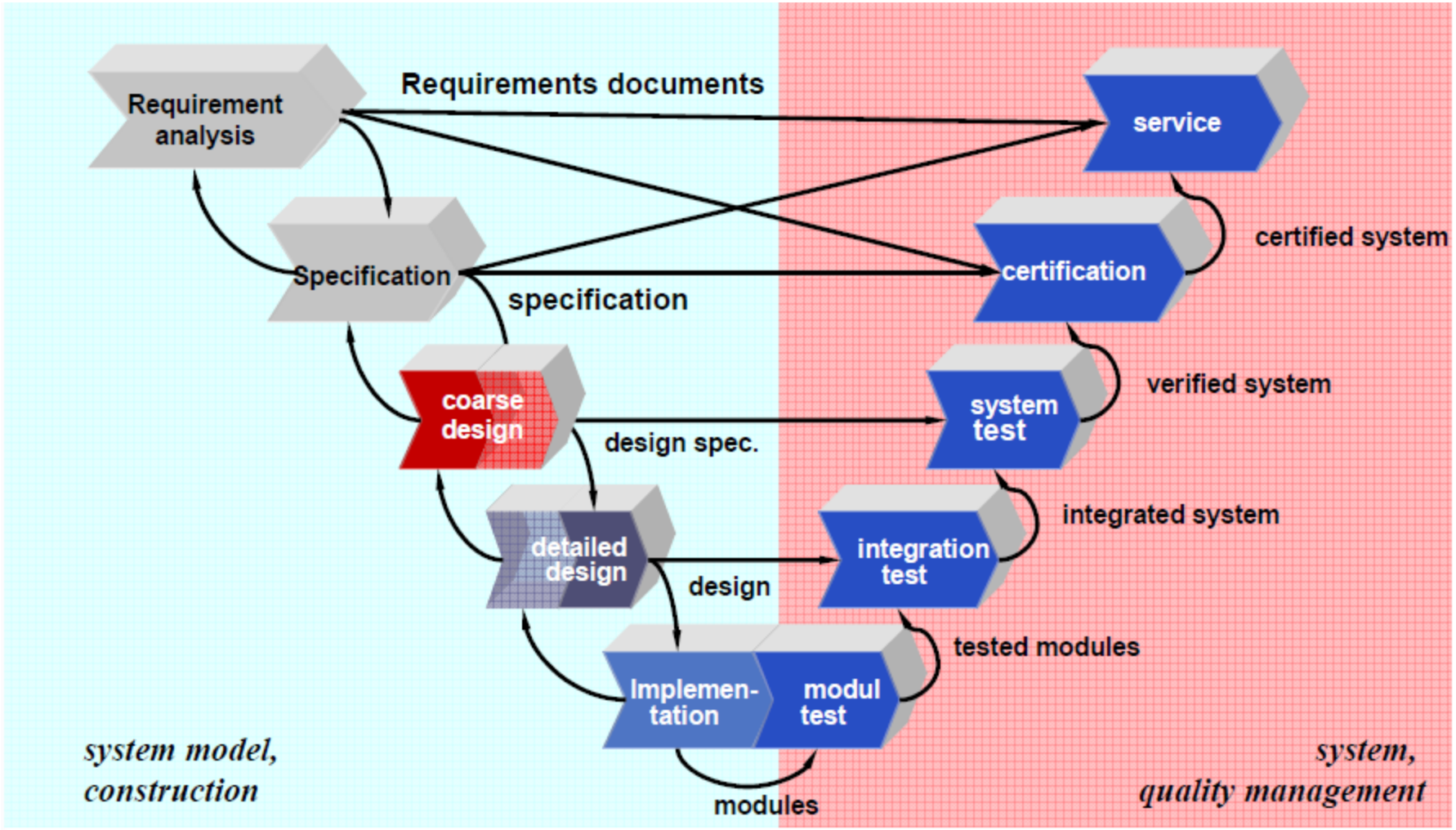
- Great when requirements are unclear
- Customer is involved in every loop
- Risk: prototype may accidentally become the final product

- Used heavily in embedded systems

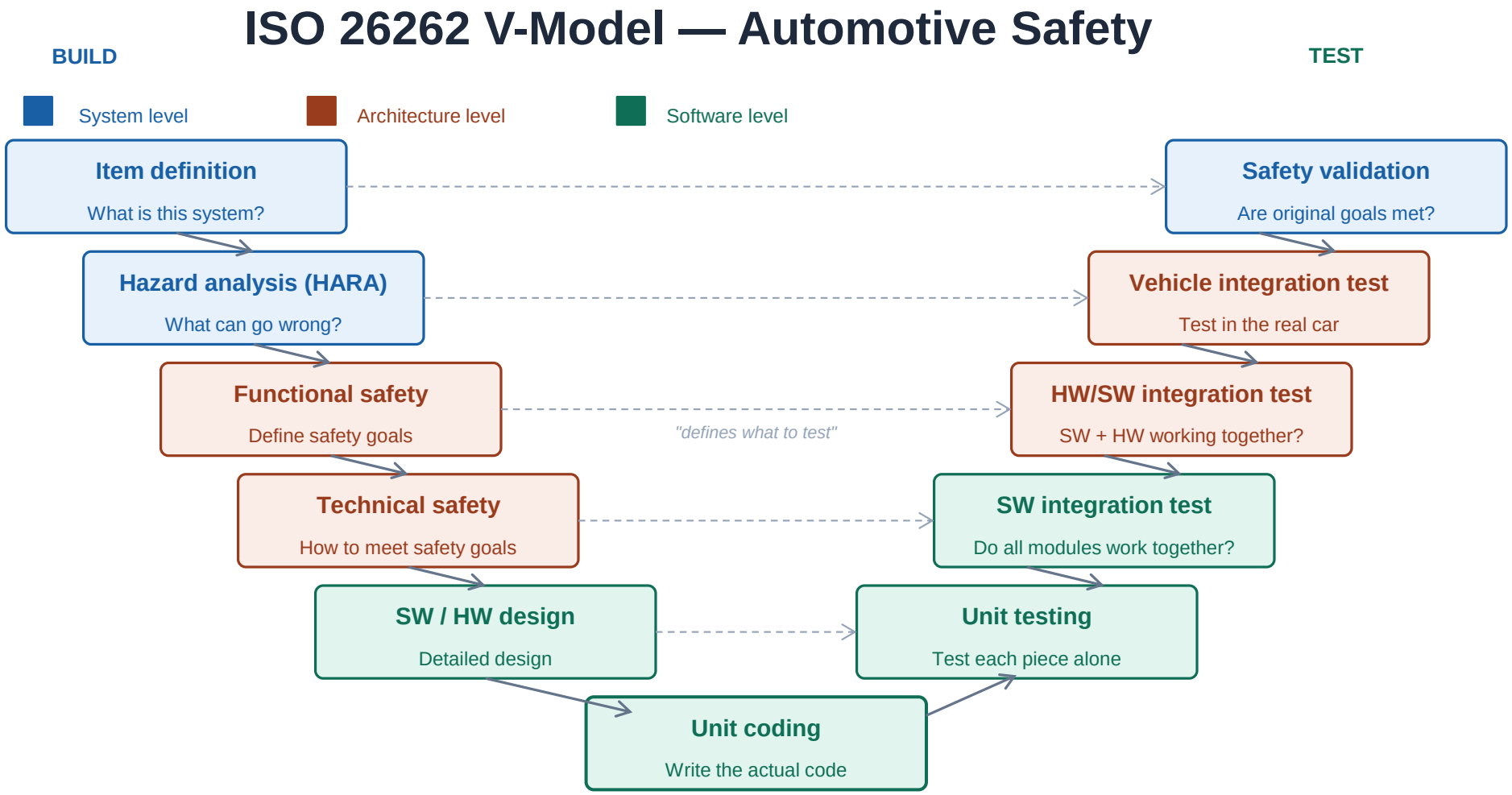
Robot

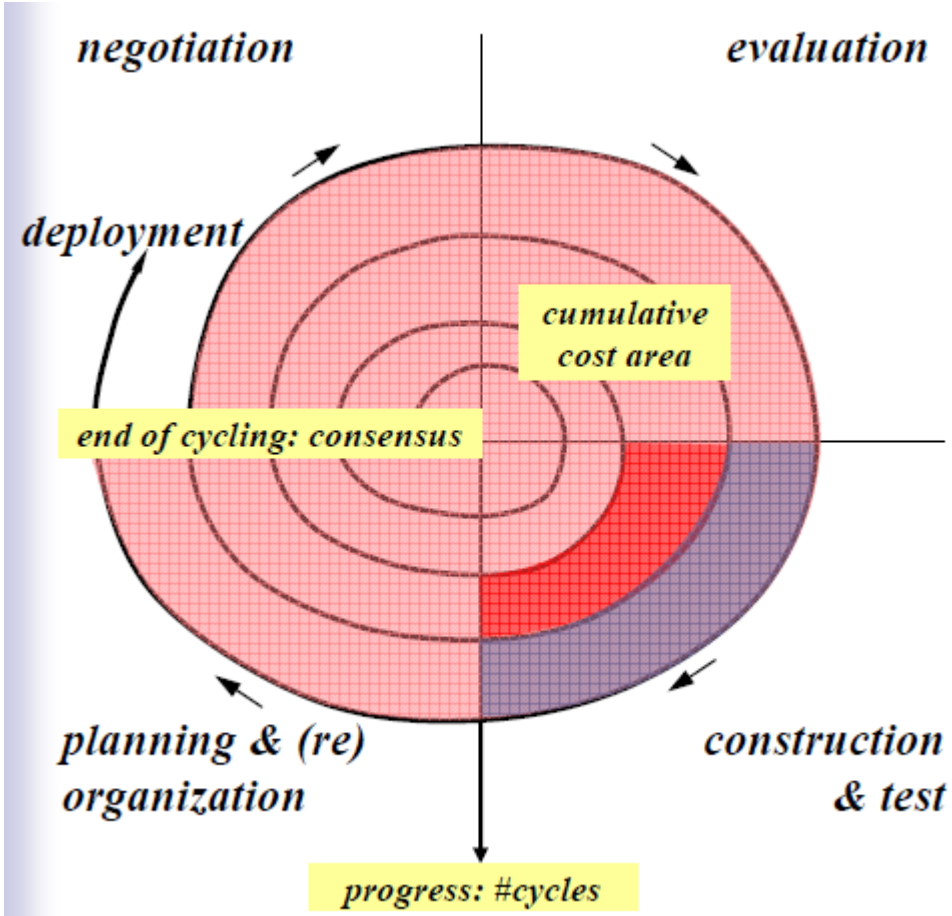
- Build rough robot Week 2,
- Test on track Week 3 (line-following too slow)
- PID → obstacle detection fails
- adjust thresholds > retest.
- Each iteration is a prototype cycle.
- Final demo Week 12 = system acceptance test.





- You Plan
- You Prove


ISO 26262 V-Model — Automotive Safety
BUILD
TEST
System level
Architecture level
Software level
Item definition
What is this system?
Hazard analysis (HARA)
What can go wrong?
Functional safety
Define safety goals
Technical safety
How to meet safety goals
SW / HW design
Detailed design
Unit coding
Write the actual code
Unit testing
Test each piece alone
SW integration test
Do all modules work together?
HW/SW integration test
SW + HW working together?
Vehicle integration test
Test in the real car
Safety validation
Are original goals met?
"defines what to test"



- Negotiation
  - objectives, alternatives, strategies, constraints
- Evaluation
  - alternatives: „Make-or-Buy“, risk analysis
- Construction & Test
  - any SE-Process for partial or full system!
- Planning
  - Review,
  - Plan next phases

Each loop produce a more complete version of the system

[Boehm1988]

HOCHSCHULE HAMM-LIPPSTADT

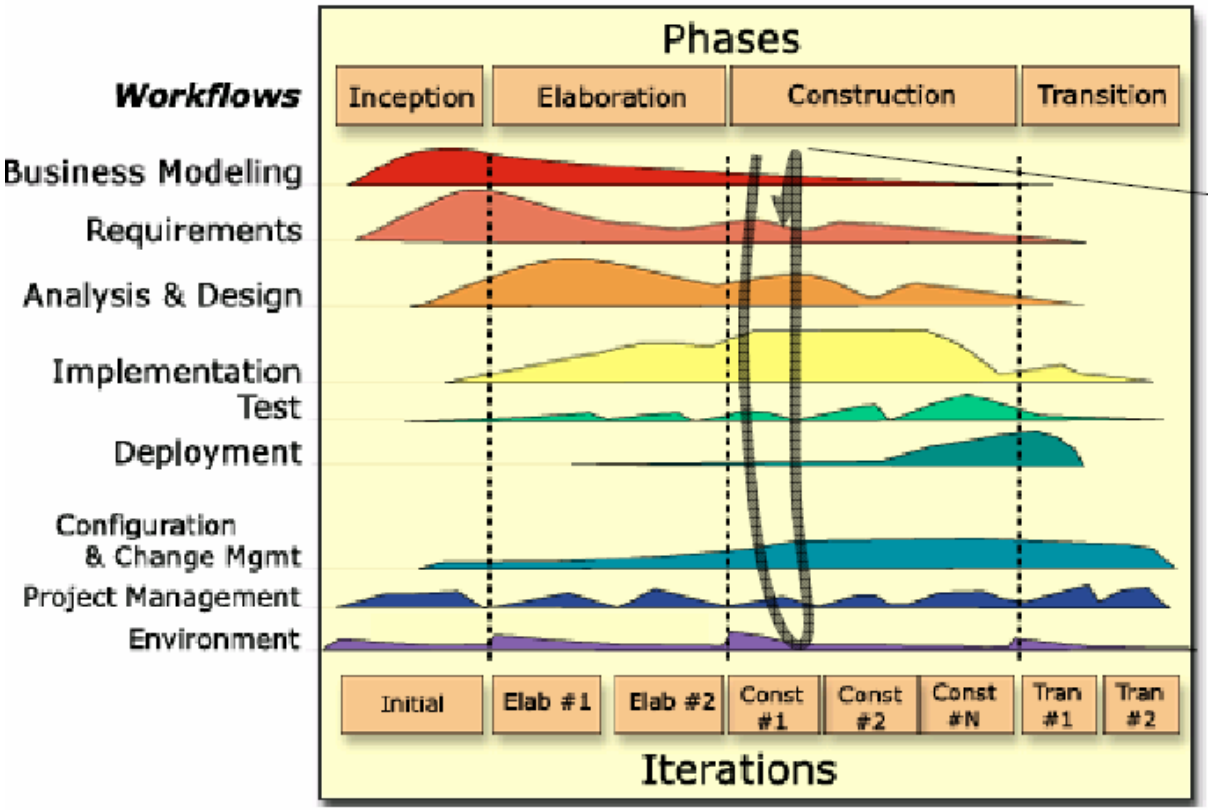
How many loops should a Spiral Model have, and what are the risks of too many?

Optimal number is **3 to 6 loops** — one loop per major risk. When no significant unknown remains, stop and build.

- **Cost** — every loop needs team time, tools, and testing budget
- **Time** — each loop takes weeks or months, delaying the final product
- **Analysis paralysis** — team keeps re-evaluating instead of building
- **Scope creep** — new features added every loop, product never finishes
- **Team fatigue** — motivation drops after too many repeated cycles
- **Diminishing returns** — after loop 5-6, no new critical risks are found

16 Rational Unified Process (RUP)

Combines iterative development with structured multiple workflows



RUP Overview Diagram

Inception: Is this Proj worth doing?

(scopes and business case)

Elaboration: Understand the architecture, biggest risks.

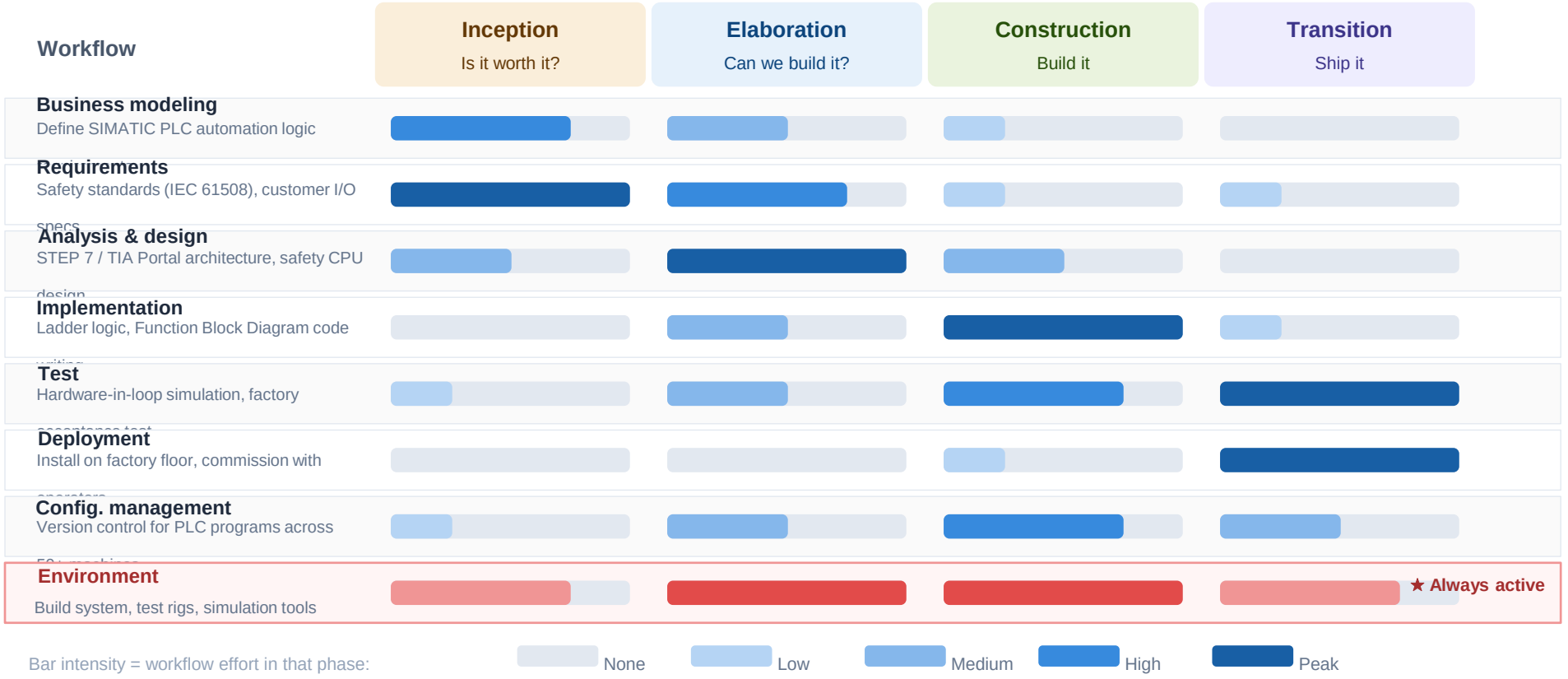
Construction: Built it across multiple iterations.

Transition: deploy to users, train, fix bugs.

- Q: When you build the robot, are you doing only hardware today, and software tomorrow? Or are you always doing both?

Rational Unified Process (RUP)

Siemens: Industrial automation software (SIMATIC)

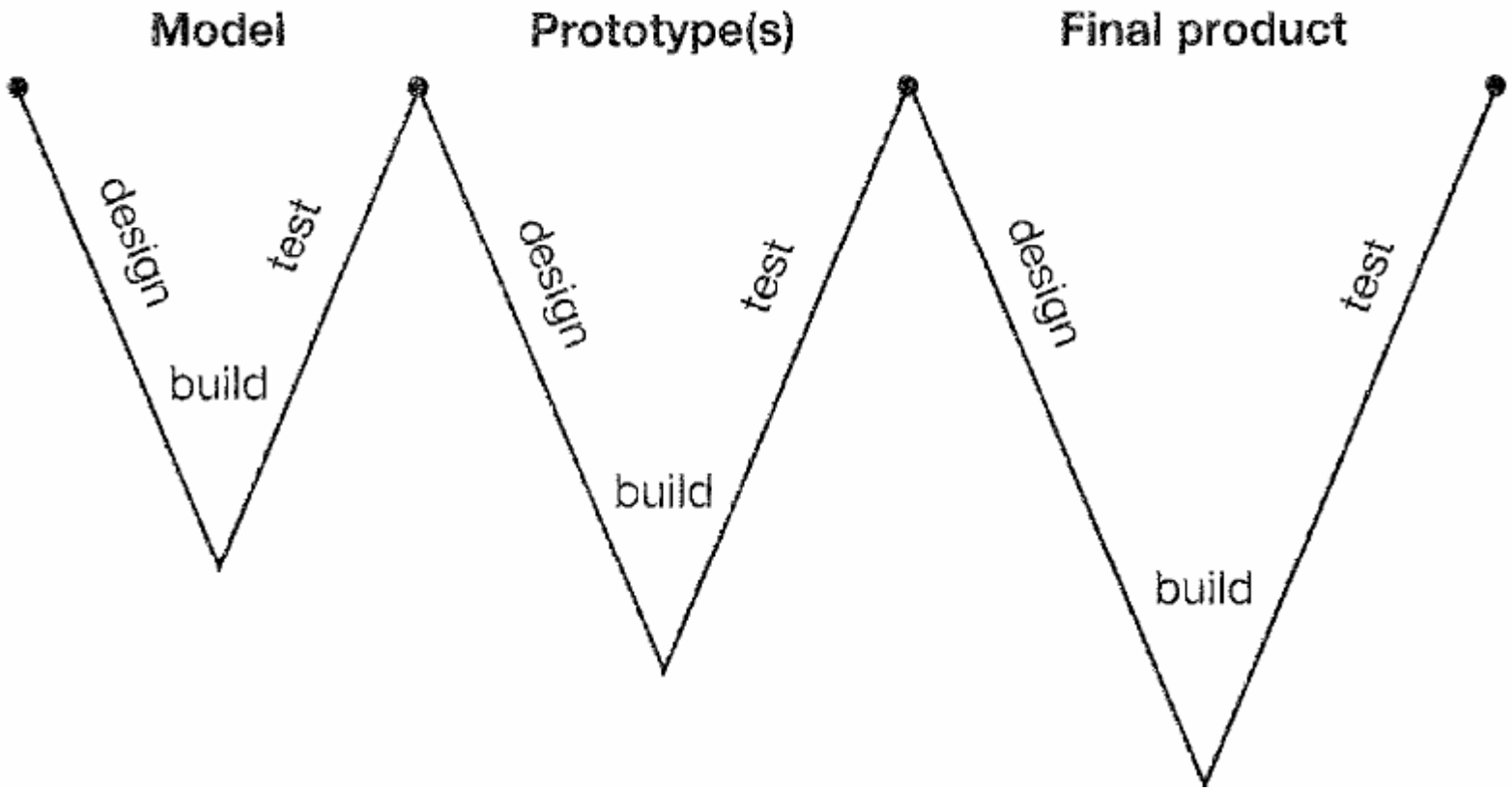


HOCHSCHULE HAMM-LIPPSTADT

1. Software Engineering Life Cycle Models
2. System Engineering Life Cycle Models
3. **Embedded System Life Cycle Models**
4. Advanced Life Cycle Models & MDD
5. Process Improvement
6. Discussion & Summary
7. Bibliography

1. 3V Model
2. Multiple V Model

201. 3 V Model (1/2)



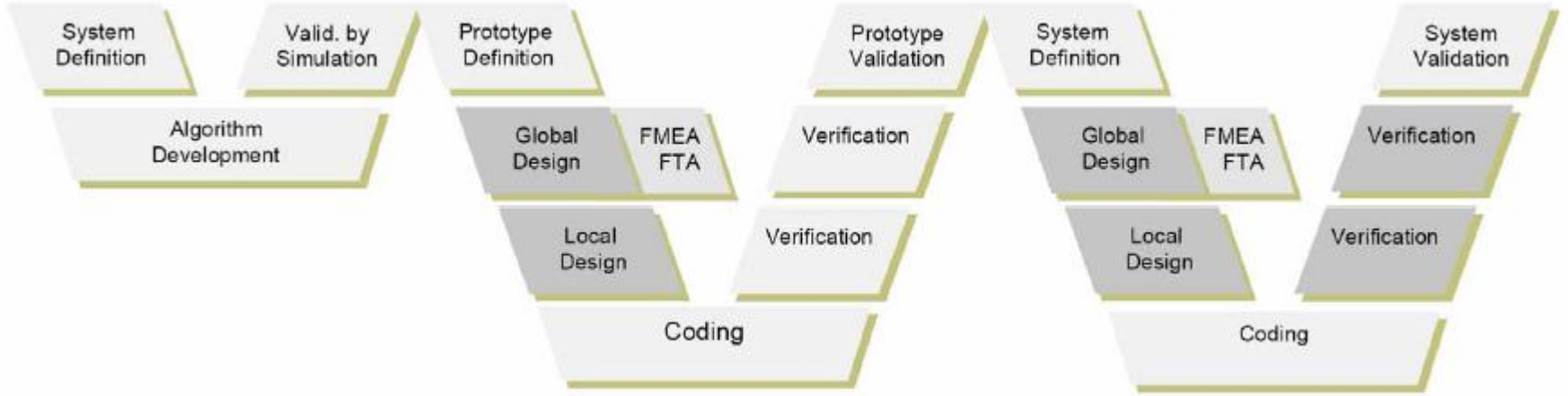
Simulation only, no HD, check algorithm logic

Real HD, task scheduling, timing

Target HD, real performance constraints

[Broekman&Notenboom2003]

21 V Model (2/2)



- Model: covers the definition and simulation of the overall system functionality
  - Implementation aspects are not considered
- Prototype: is characterized by rapid prototyping
  - hardware specific parameters become important
  - deployment & message scheduling
  - local design addresses the scheduling of tasks on each node
- Final product: addresses the system development for the final target hardware
  - typical problem: limited performance of the target system

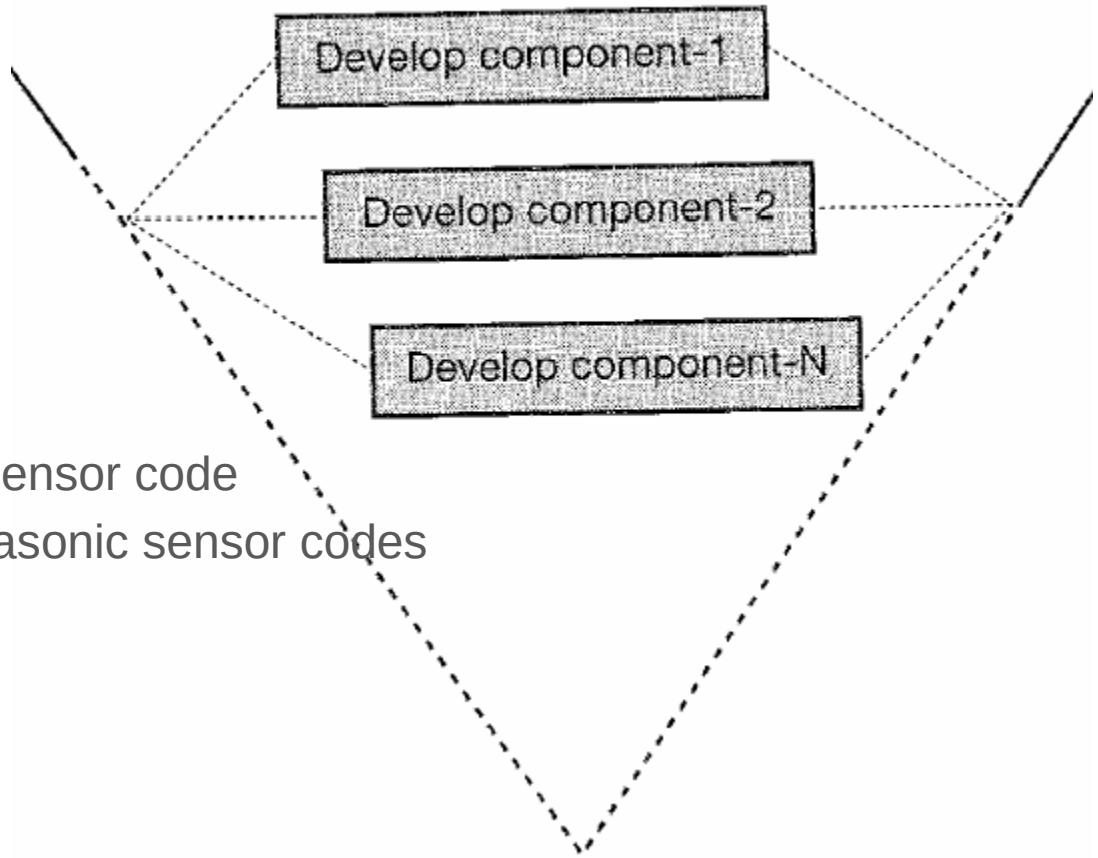
http://www.vmars.tuwien.ac.at/projects/setta
http://www.vmars.tuwien.ac.at/projects/setta/docs/meetings/020121p/final_document.pdf

V Model (2/2)

- Q: What happens if your IR sensor gives a wrong reading? Does the robot crash, stop, or ignoring it?

- V1: Tinkercad Simulation: never happens, simulation is perfect,
- V2: Real circuit on a breadboard: robot might crashes or stop randomly, it gets confused by a shiny surface
- V3: Put everything inside the robot chassis: motors creates ele noise that disturbs the IR sensor

[Broekman&Notenboom2003]

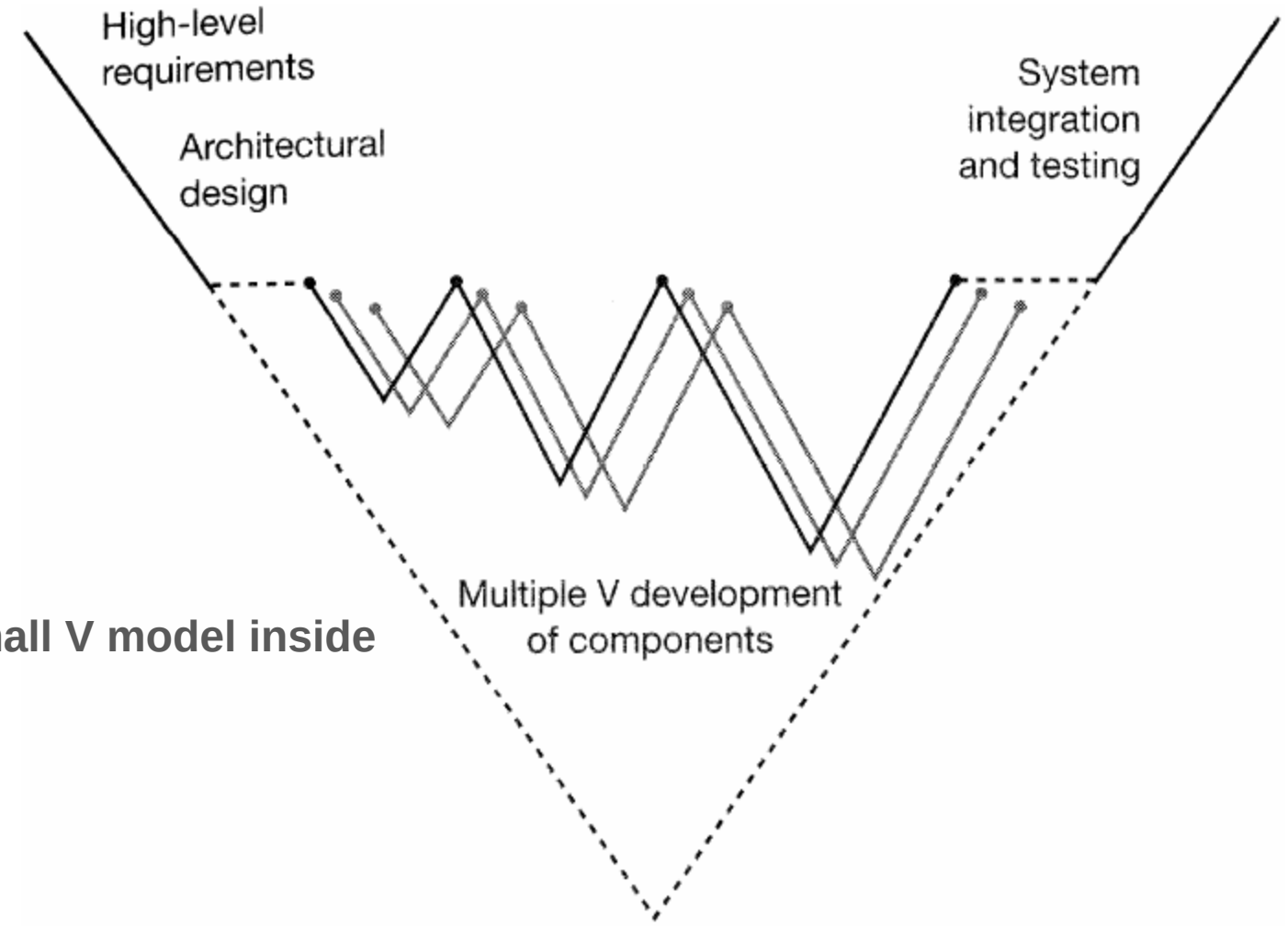


- Person 1: IR sensor code
- Person 2: Ultrasonic sensor codes
- …..

Integration moment

biggest risk: everything combined into one system

24 Multiple V Model (2/2)

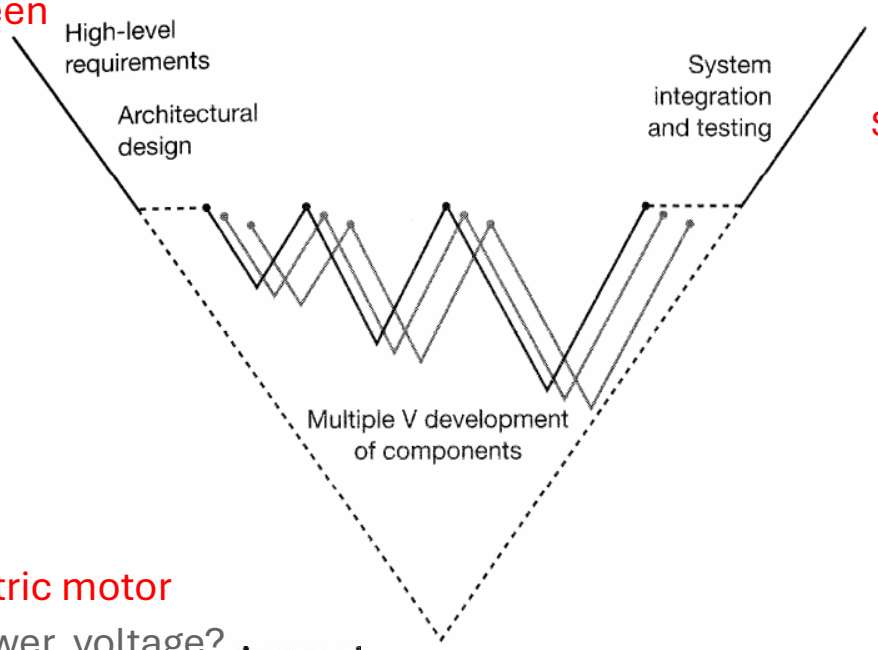


Many small V model inside

[Broekman&Notenboom2003]

25 Multiple V Model, HSUV

1) "SUV must drive 800km on one tank, switch between electric and petrol automatically"



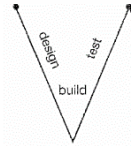
2) full SUV test on real road, highway, city, mountain

System integration: hardest phase

Team 1) Battery & electric motor

Design: how much power, voltage?

Code: bttery mngt SW,



Test: does battery charge and discharge correctly at 20-40?

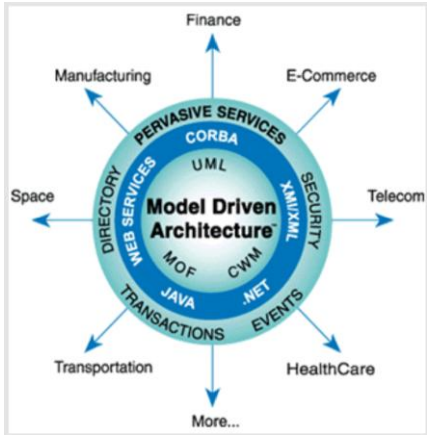
Team 2) Petrol engine & transmission

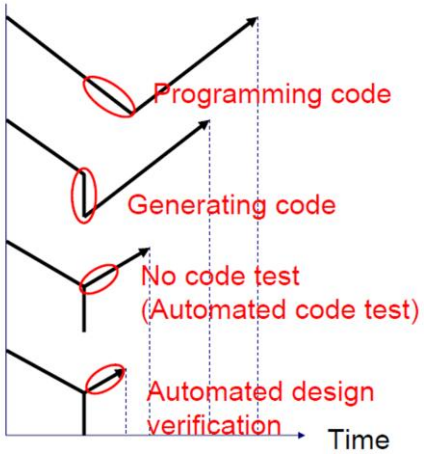
Team 3) Hybrid controller,logic for switching between electric and petrol

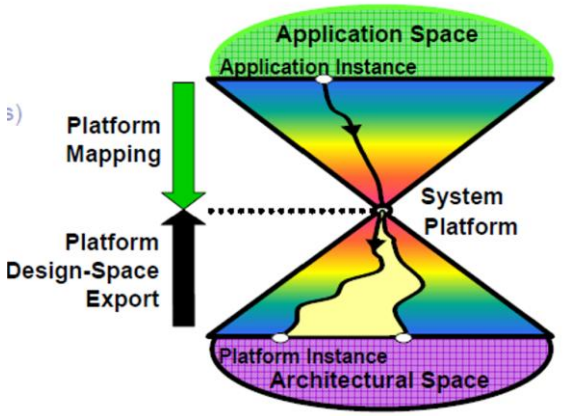
HOCHSCHULE HAMM-LIPPSTADT

1. Software Engineering Life Cycle Models
2. System Engineering Life Cycle Models
3. Embedded System Life Cycle Models
4. **Advanced Life Cycle Models & MDD**
5. Process Improvement
6. Discussion & Summary
7. Bibliography

1. MDA
2. Y-Model
3. Platform-Based Design

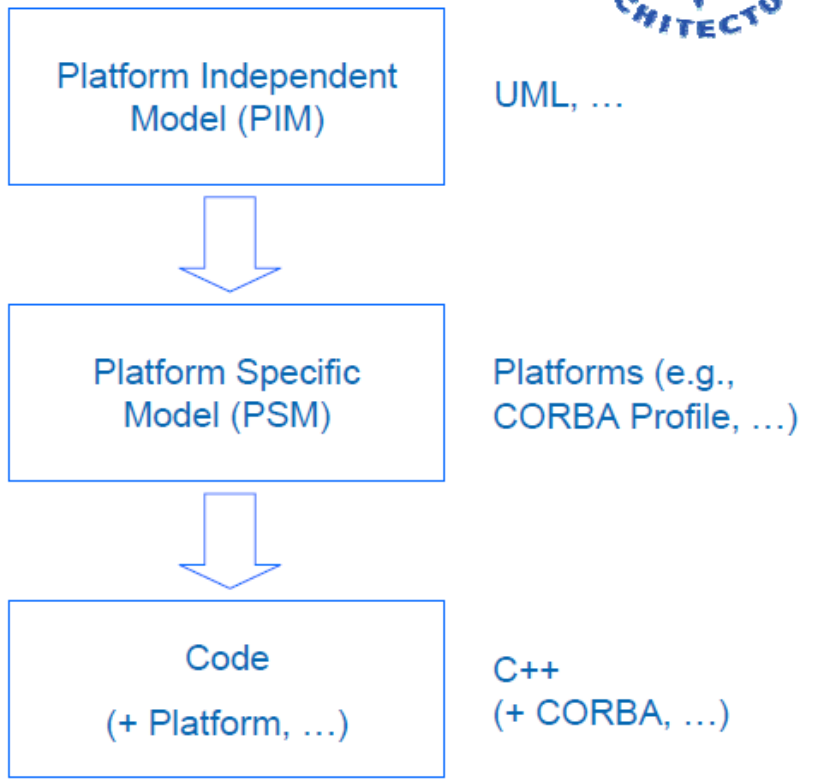




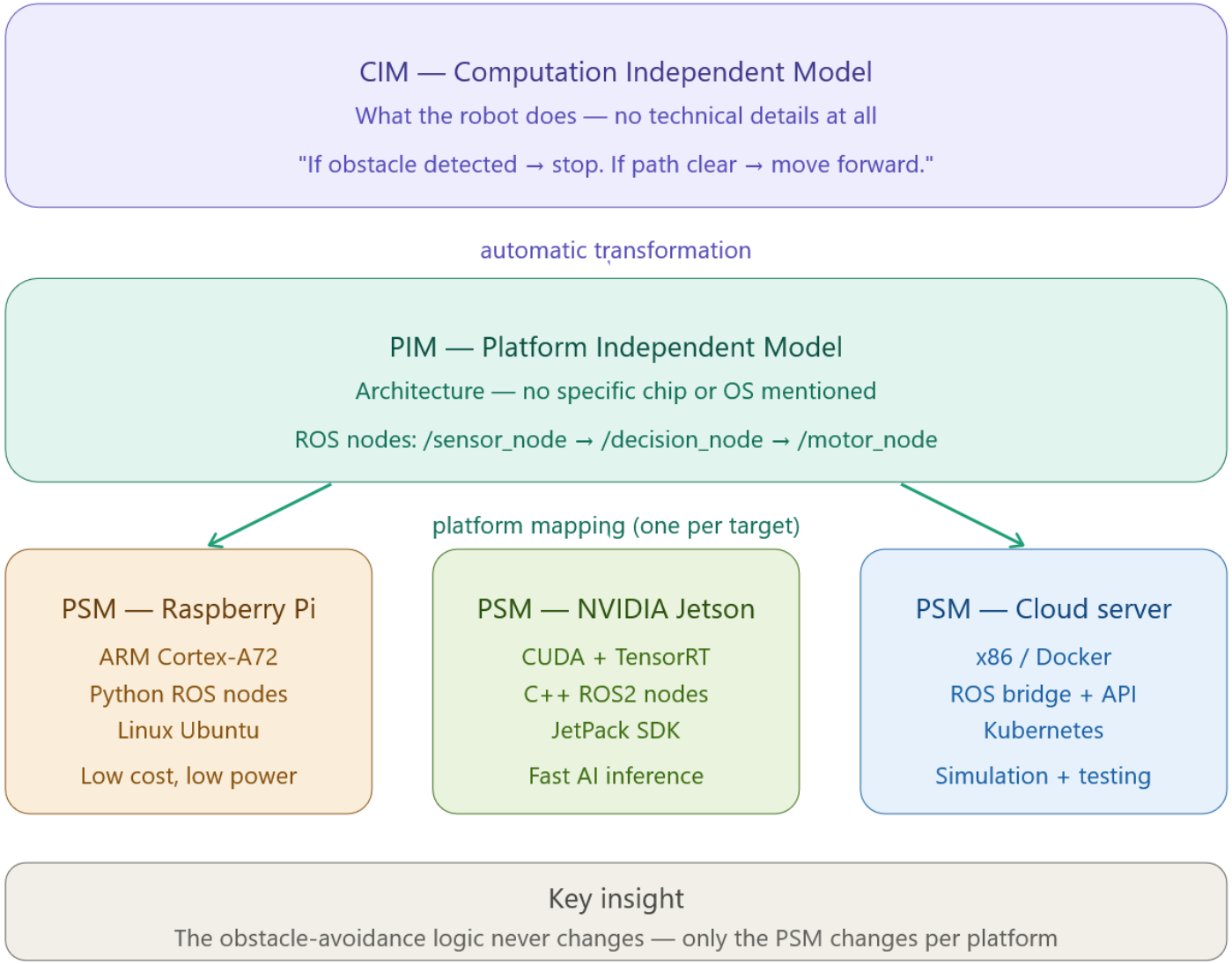


Model Driven Architecture, an approach from OMG

- An approach to IT system specification that separates **what a system does** from **how it runs on specific HD, or SW platforms**.
- "Design **once at an abstract level**, automatically transform it to work on **any** platform"
- PIM: describe functionality (no technology)
- PSM: transform to a specific platform (.Net, Java,)
- Code: Excitable code for target platform

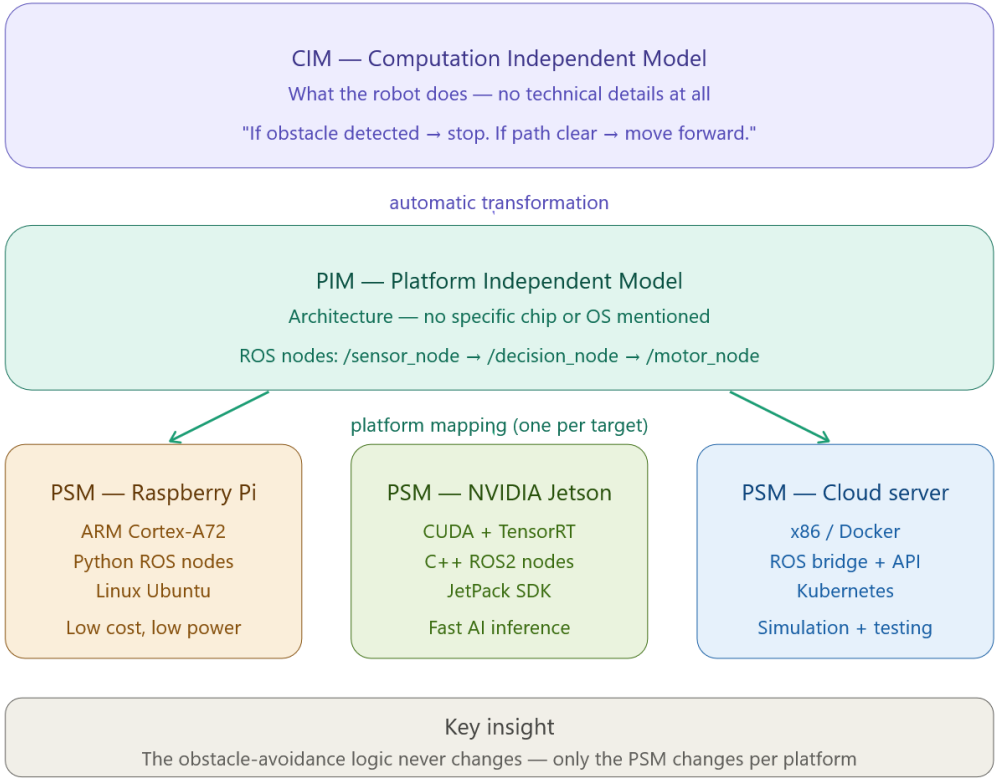


29 1. MDA: vehicle robot



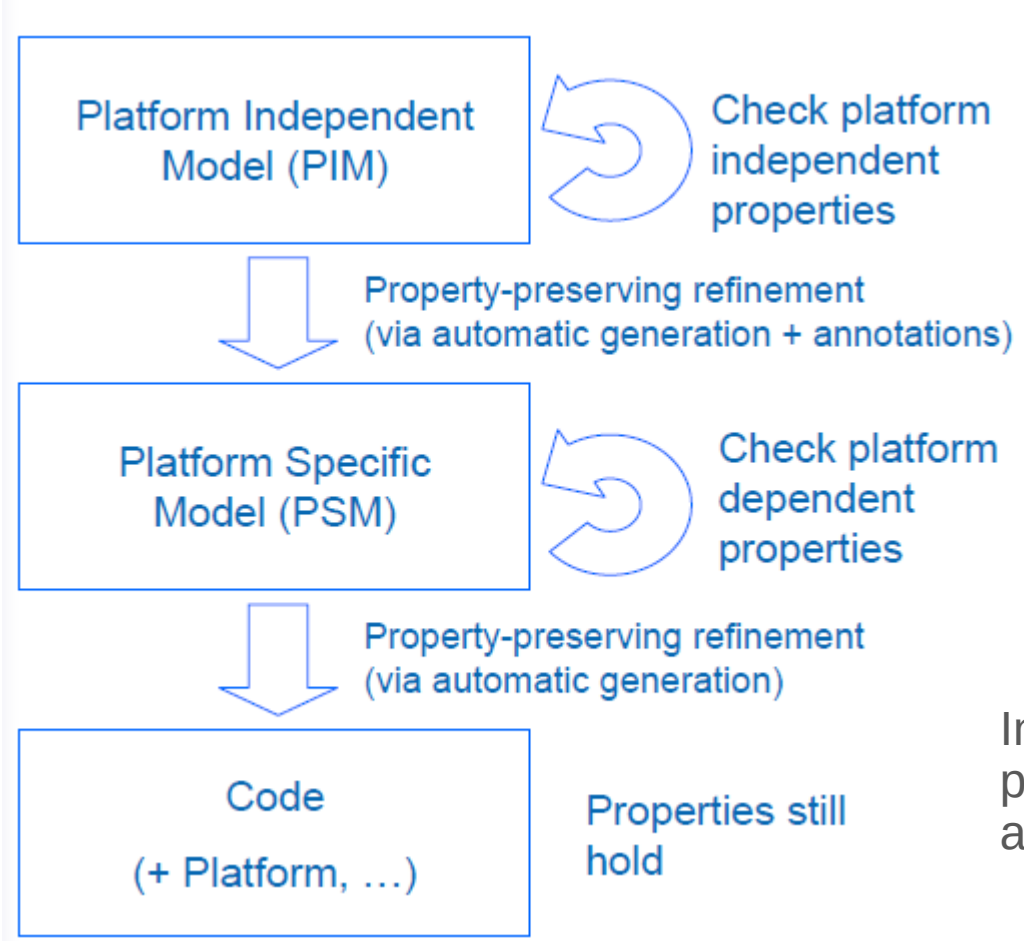
- CIM: think like a user not an Engineer
- PIM: think like a software architect. No mention of raspberry Pi, it works on any platform
- PSM: pick the platform

30 1. MDA: vehicle robot

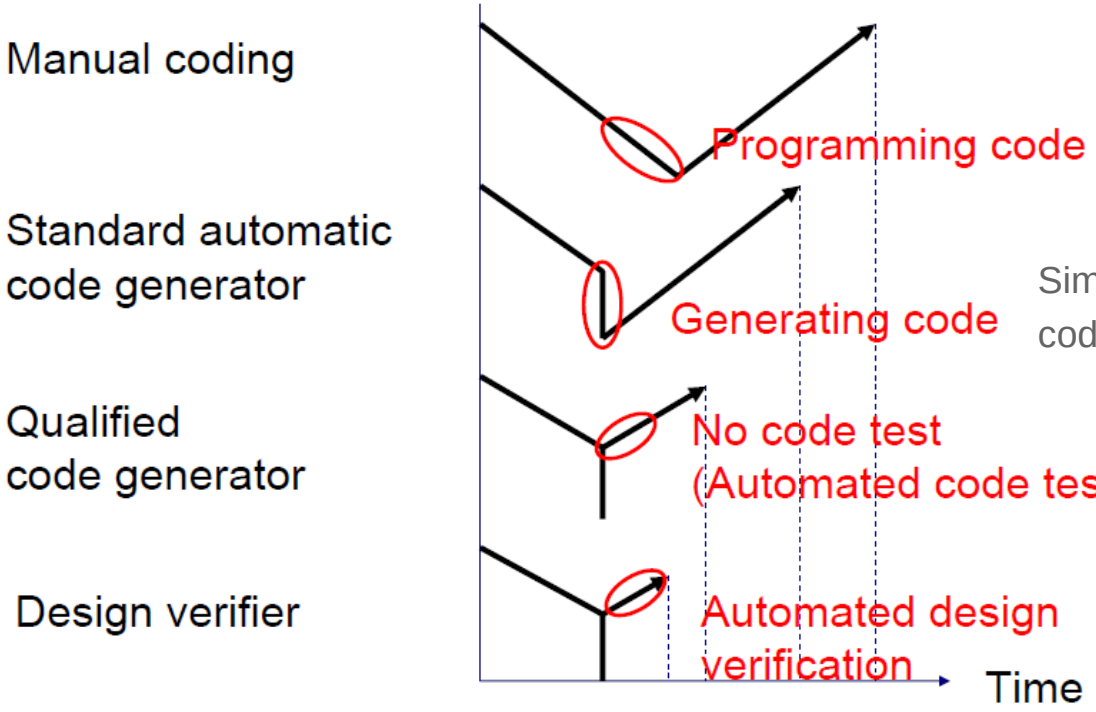


- **Without MDA**: If you wrote the code directly for the Arduino, and someone asks to run it on Jetson nano, you have rewrite everything.
- **With MDA**: rewrite only the PSM layer not the logic.

- We can verify properties of the system at the model level before any code!



Traceability: always trace code back to the req, **Portability**: model can generates code for multiple platforms



Simulink Code Generate code, code must be tested manually.

Tool is certified, generated codes need minimal additional testing.

Formal mathematical verification at design level, code testing unnecessary.

Progress in code development tools, human effort decreases and tool quality requirements increase.

Application Example from Airbus

- **Tool:**
  - Safety Critical Applications Development Environment (SCADE)
- **Application:**
  - A340/600 FCSC (Flight Control Secondary Computer):
- **Result:**
  - 70 % automatically generated code
  - 50 % reduction in development cost
  - reduction in modification cycle time by factor 3

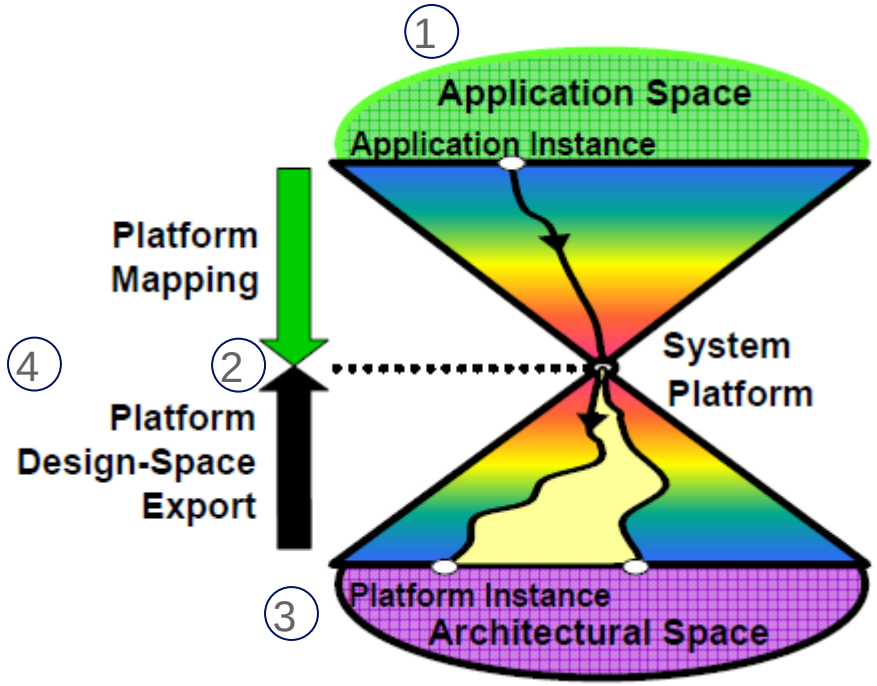


Source: Esterel Technologies

3. Platform based design

Hourglass structure

It separates what system does from what hardware it runs on it.



1) All the possible behaviors the system could have

2) 2) a common interface between SW and HD

3) 3)all possible hardware configurations available

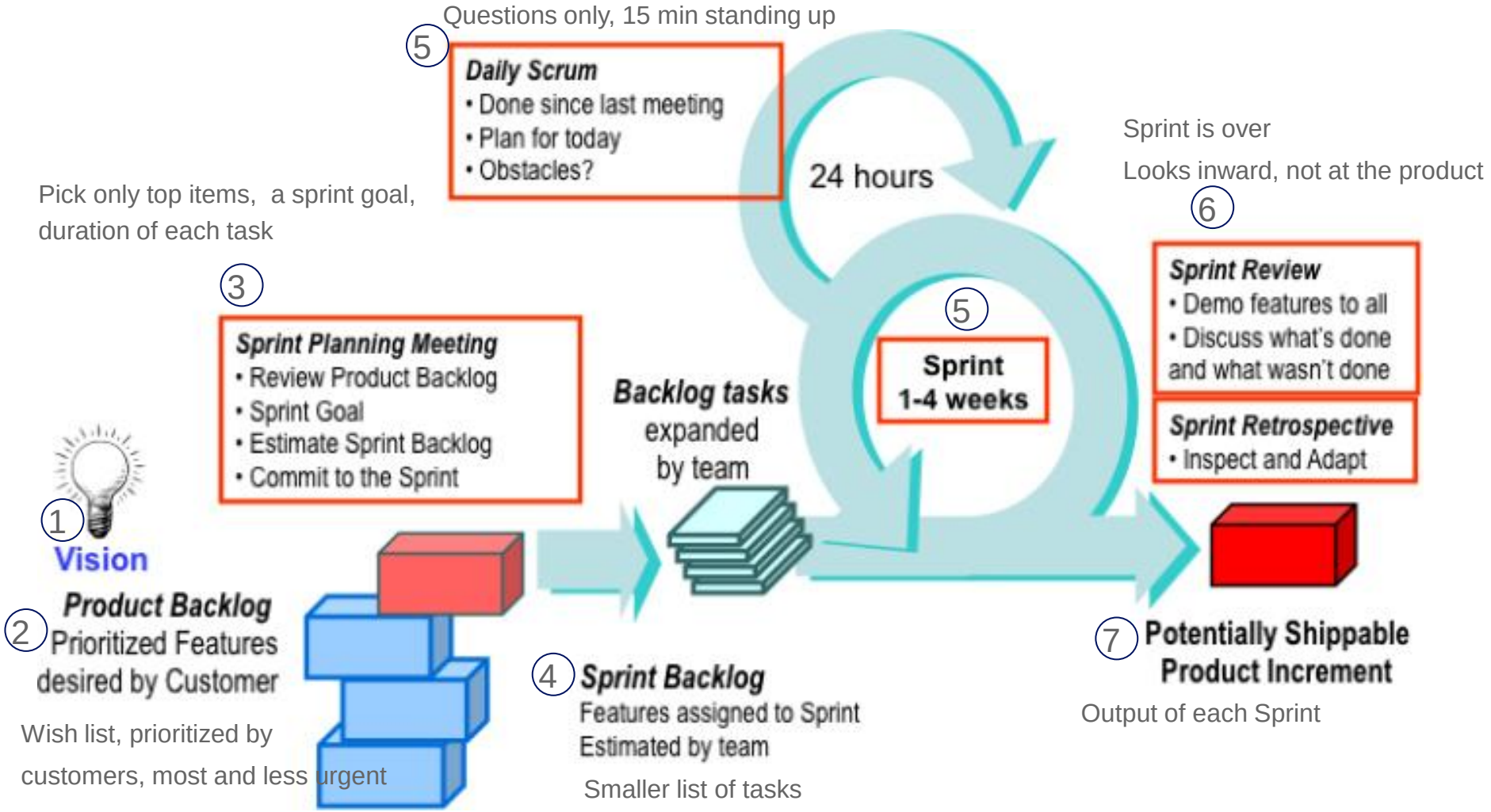
4) Top down: start from app requirements and map downward onto platform constraints

5) Bottom-up: start from available HW and export its capabilities upward

HOCHSCHULE HAMM-LIPPSTADT

- **Platform:**
  - a family of architectures satisfying a set of constraints imposed to allow the reuse of hardware and software components.
- **Platform-based design:**
  - meet-in-the-middle approach: In the top-down design flow, designers map an instance of the upper platform to an instance of the lower, and propagate design constraints.

Agile - Scrum



https://www.pmi.org/learning/library/agile-project-management-scrum-6269

Scrum in practice — Spotify

Small squads, 2-week sprints, millions of users updated continuously

Spotify squads

Each squad owns one feature forever

Squad A
Discover Weekly

Squad B
Podcast recs

Squad C
Search

Squad D
Radio feature

6-8 people each · independent · run own Scrum

2-week sprint cycle

Sprint planning
Pick tasks from backlog. Set sprint goal.

↓

Daily scrum
15 min every morning. Done? Doing? Blocked?

↓

Sprint (2 weeks)
Team builds. Squads work independently.

↓

Sprint review
Demo to stakeholders. Check data. Skip rate up or down?

↓

Retrospective
How did we work? What to improve next sprint?

Output every sprint

New Discover Weekly algorithm deployed to millions of users

Not 10 test users.

Real users. Immediately.

If something breaks

Waterfall: wait 2 years for next big release

Scrum: fixed in next sprint (2 weeks)

https://www.pmi.org/learning/library/agile-project-management-scrum-6269

1. Software Engineering Life Cycle Models
2. System Engineering Life Cycle Models
3. Embedded System Life Cycle Models
4. Advanced Life Cycle Models &
5. Process Improvement
6. Discussion & Summary
7. Bibliography

III The Development Life Cycle

1. Software Engineering Life Cycle Models
2. System Engineering Life Cycle Models
3. Embedded System Life Cycle Models
4. Advanced Life Cycle Models &
5. Process Improvement
6. Discussion & Summary
7. Bibliography

[Boehm1988] Barry W. Boehm. A Spiral Model of Software Development and Enhancement. IEEE Computer, 21(5):61 72, 1988.

[Camus&Dion2003] Jean-Luis Camus and Bernard Dion. Efficient development of airborne software with scade suite. 2003.

[Galin2004] D. Galin, Software Quality Assurance: From theory to implementation. Harlow, England: Pearson Addison Wesley, 2004.

[RUP1999] Ivar Jacobson, Grady Booch, and James Rumbaugh. The Unified Software Development Process. The Addison-Wesley Object Technology Series. Addison-Wesley, January 1999.

[Sage&Armstrong2000] Andrew P. Sage and James E. Armstrong. Introduction to Systems Engineering. Wiley Series in Systems Engineering and Management. Wiley-Interscience, March 2000.